Pricing Liquidity on Real World Assets in DeFi

M.S.C. Hernandes*, Ph.D.
IPOR Labs

March 2026

Abstract

We develop a utility-based framework for pricing liquidity in tokenized real-world assets (RWAs) with delayed redemption in decentralized finance (DeFi). The central problem is whether an investor should sell immediately in the secondary market or wait through the redemption cycle for an uncertain future payoff. The liquidity question is useful in the case of early redemption (looping or liquidity by fund), or risk mitigation (non-performing loans in private credit, macro event, etc.).

We formalize this choice as a comparison between the spot sale value and the certainty equivalent liquidity of waiting, which makes risk preferences explicit and yields a natural pricing rule for illiquidity.

Using exponential utility and stochastic asset models, including geometric Brownian motion and two-sided jump–diffusion, we show how volatility, tail risk, and model choice affect liquidation decisions. We also outline a modular pricing-engine architecture for estimating liquidity discounts and managing redemption risk in tokenized RWAs.

Keywords: DeFi, real-world assets, tokenization, liquidity, utility theory, certainty equivalent, stochastic pricing.

1 Introduction

A central challenge in Decentrilized Finance (DeFi) markets for tokenized real-world assets (RWAs) is that many such assets have *delayed redemption*, i.e., assets with redemption delays of days or weeks (e.g., see [1, 2, 3]). When an investor holding a RWA token with delayed redemption needs to exit their position, at the best case scenario they will be faced with the following **liquidation decision problem**:

1. Sell the RWA token at spot rate offered in the market (held by market makers, or available DEXs), likely at a discount and with slippage, but receiving a sure value today.

2. Wait until the redemption cycle, for an expected payoff in the future.

Because the price of the asset at redemption time is uncertain, we call its price a stochastic process, and we interpret its expected payoff as the average amount to be redeemed if we were to repeat this decision multiple times.

In short, market participants dealing with RWAs with delayed redemption need to solve a stochastic decision problem when deciding to buy/sell tokens. These notes propose a pricing engine to efficiently solve this problem.

*quants@ipor.io

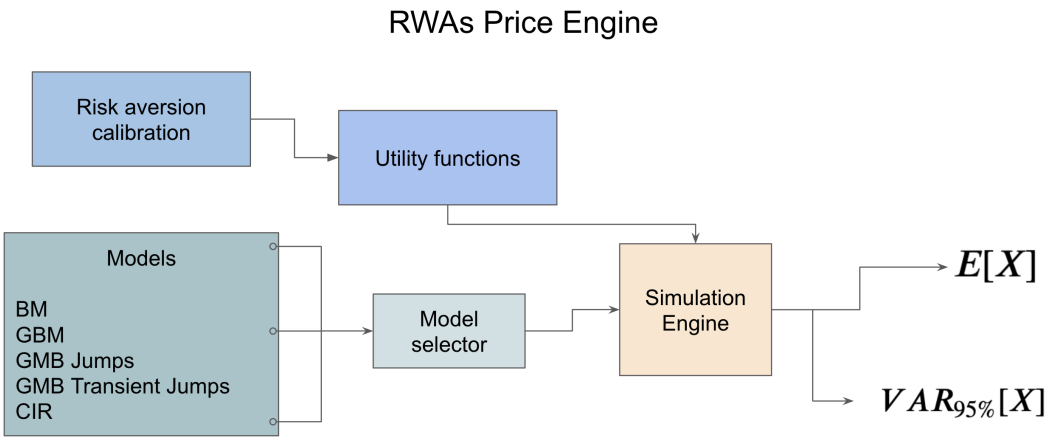


Figure 1: Pricing engine diagram

1.1 Our Framework

In these notes we describe a pricing engine to support an investor holding RWA token facing the liquidation decision problem. In broad strokes, the proposed price engine has the following components (see figure 1 for a diagram of the architecture):

1. **Risk aversion calibration**: estimate investor risk preferences from observed behavior or stated utility.
2. **Utility functions**: rules describing how investors value uncertain payoffs.
3. **Asset models**: stochastic models for the underlying asset over the redemption horizon (Geometric Brownian motions, Jump difussion, etc.).
4. **Simulation engine**: compute expected utility of the underlying asset.

   The code in appendix A shows an example of how the pricer can be used with *Dope*[1].

1.2 Use case examples

In this section we briefly go over an example to illustrate the motivation and the power of our framework.

In the following example, we present two scenarios where all variables are kept constant except for the volatility of the underlying asset. If our framework works as intended, we should see the higher volatility asset to yield a lower utility value from our pricer. That is exactly what we will observe.

Example 1.1 (Volatility impact on pricing)**.** Figure 2 shows the historical price of sUSDai, a staked, yield-bearing token, "withdrawable after an unstaking period" (from [4]) of 30 days, i.e., the redemption delay is variable from 0 to 30 days, depending on the time to the next redemption window.

While sUSDai is not a RWA strictly speaking, this example can easily be extrapolated to typical RWAs.

Inspired by the data in figure 2, let's compute the utility value of two RWAs using our pricer, with the same parameters except for the volatility of the underlying asset for both of them. Table 1 summarizes the parameters for two cases with the following parameters:

[1] https://github.com/ipor-labs/dope

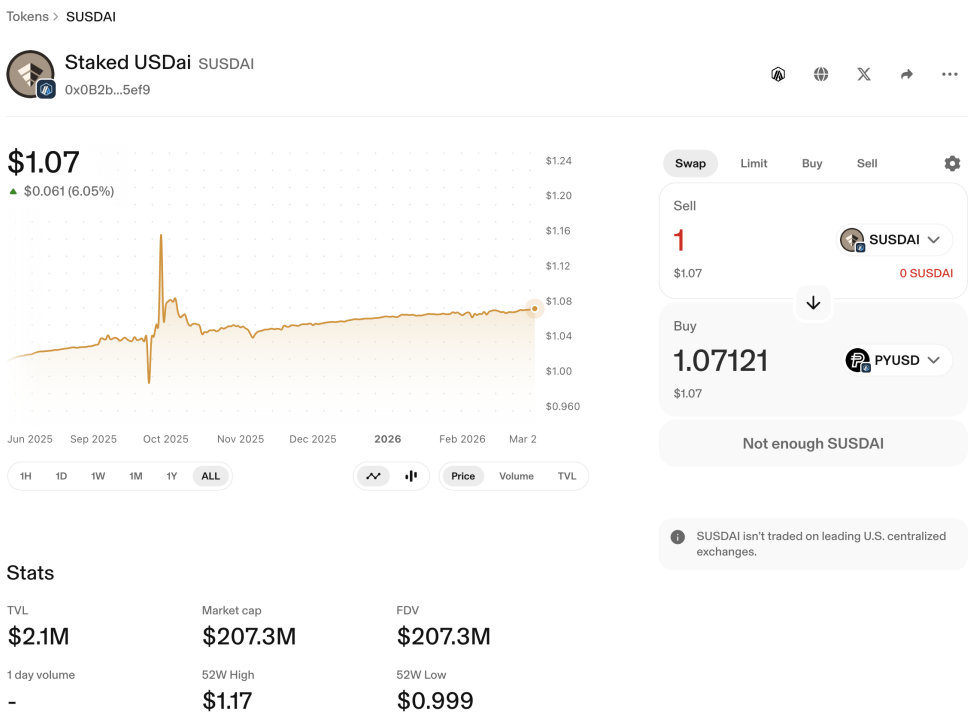


Figure 2: Historical sUSDai price data from Uniswap

	Parameters						
Case	V_0	σ_{daily}	μ_{daily}	Risk Aversion	Redemption	X_{sell}	$\mathbb{L}[X_{\text{wait}}]$
1	1.07	0.15%	0.1%	γ	30 days	1.064	1.097
2	1.07	0.60%	0.1%	γ	30 days	1.064	1.019

Table 1: Pricing example. If two assets are almost identical, with only the difference between them being the volatility, our proposed pricer evaluates the more volatile asset at a lower value than the less volatile asset. In this case, triggering a spot sell instead of waiting for redemption. This is one of the expected properties of a pricer, that can evaluate uncertainty accordingly.

- V_0 is the price of the token in the market right now.
- σ_{daily} is the daily volatility of the underlying asset.
- μ_{daily} is the daily return trend of the underlying asset.
- Aversion parameter is a parameter that captures how risk averse the investor is. We discuss the parameter γ in more details later.
- X_{sell} is the value obtained by selling the RWA token immediately in the market (with 50 bps slippage when selling on the spot market).
- $\mathbb{L}[X_{\text{wait}}]$ is our pricer output, representing the expected value obtained by waiting through the redemption cycle. We will interpret and discuss how to compute that value later.

See how case two (higher volatility), yields a value for $\mathbb{L}[X_{\text{wait}}]$ lower than X_{sell}. A more volatile asset, receives a lower expected value from our price. Which is exactly the expected behavior of

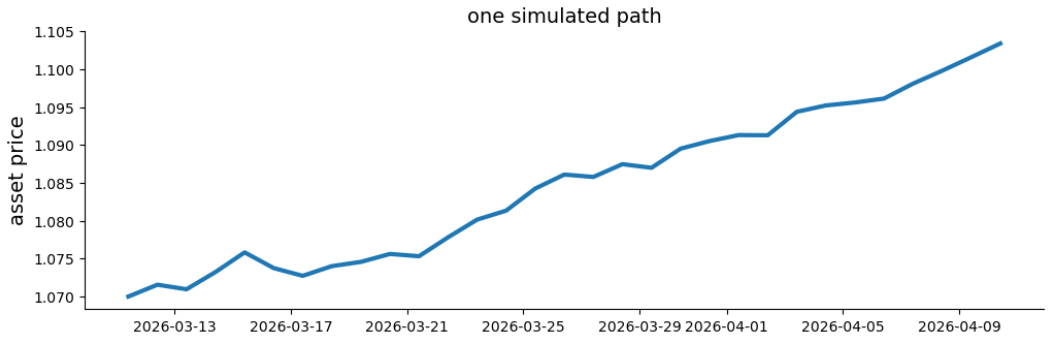


Figure 3: Simulated path for a stochastic asset with $\mu = 0.1\%$ and $\sigma = 0.15\%$

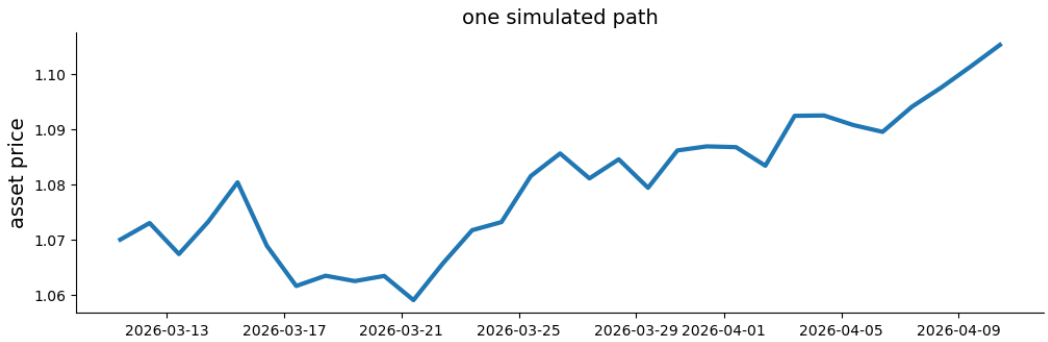


Figure 4: Simulated path for a stochastic asset with $\mu = 0.1\%$ and $\sigma = 0.60\%$

a pricer that can price uncertainty accordingly: Everything being equal, a risk averse investor should value a riskier asset lower than a less risky asset.

Figures 3 and 4 display one path simulated (per case) as a stochastic process with the parameters from table 1.

This example shows that our pricer can capture the impact of volatility on the expected value of waiting for redemption. This is one of many key features of a pricer.

For the remaining of the notes, we will focus on the theoretical framework and the architecture of the pricer.

2 Liquidation Decision Problem as a Stochastic Problem

As discussed in the introduction, a key friction for RWAs is delayed redemptions: assets may trade onchain, but redemption of the underlying claim is often scheduled and delayed. This forces investors to a decision when trying to decrease exposure. We call the **liquidation decision problem**:

- Sell the RWA token at spot rate (often a discount) now; or
- Wait for an uncertain payoff at redemption time.

We formalize this stochastic decision problem in this section as follows:

Definition 2.1. Throughout these notes we will use the following notation:

- Let V_0 be the current market price of the RWA token.

- Let $X_{\text{sell}} = V_0 \cdot (1 - s)$ be the value obtained by selling the RWA token immediately in the market, after accounting for fees and slippage s.
- Let $X_{wait} = V_T \cdot (1 - f)$ be the random payoff received if the investor waits through the redemption period. Here V_T is the (uncertain) price of the RWA token at redemption, and f represents fees or slippages applied at redemption.

2.1 Decision rule under utility

A naive benchmark for our decision problem would be to compare the dollar value of the immediate sale (X_{sell}) against the expected value of waiting ($\mathbb{E}[X_{\text{wait}}]$):

$$X_{\text{sell}} \quad \text{vs.} \quad \mathbb{E}[X_{\text{wait}}].$$

However, this comparison assumes that the investor does not care about risk, which is not realistic. For example, when faced with the choice between a sure payoff of \$100 and a 99.9999% chance of winning \$100.01 (one more cent than the sure payoff), a risk-averse investor would likely prefer the sure \$100 instead of having a 0.0001% chance of not winning. While these two options have the same expected payoff, an investor making this choice needs to consider the extra utility they get from the extra \$0.01.

In general, the uncertainty of X_{wait} must be valued through investor risk preferences.

Definition 2.2. Let $U : \mathbb{R}_+ \to \mathbb{R}$ be a strictly increasing utility function.

An investor prefers to sell immediately rather than wait if the utility of the immediate sale is greater than the expected utility of waiting, i.e., if

$$U(X_{\text{sell}}) > \mathbb{E}[U(X_{\text{wait}})]\,.$$

While this rule is useful, we will transform it to help with our intuition. Instead of computing the value $U(X_{\text{sell}})$, we will apply a transformation to $\mathbb{E}[U(X_{wait})]$ and focus on the certainty equivalent liquidity:

Definition 2.3. For any payoff X, define its *certainty equivalent liquidity* under utility U by

$$\mathbb{L}[X] := U^{-1}(\mathbb{E}[U(X)])\,,$$

whenever the inverse is well-defined.

Proposition 2.1. *Let U be strictly increasing utility function, then*

$$U(X_{\text{sell}}) > \mathbb{E}[U(X_{\text{wait}})] \text{ if and only if, } X_{\text{sell}} > \mathbb{L}[X_{\text{wait}}].$$

Hence the investor sells immediately if the selling value X_{sell} exceeds the certainty equivalent liquidity of waiting ($\mathbb{L}[X_{\text{wait}}]$).

Proof. The proof can be done directly using the fact that U is strictly increasing, which implies that U^{-1} preserves order. That means that $U(X_{\text{sell}}) > \mathbb{E}[U(X_{\text{wait}})]$ if and only if $X_{\text{sell}} > U^{-1}(\mathbb{E}[U(X_{\text{wait}})])$. By definition of certainty equivalent liquidity, we have that

$$U(X_{\text{sell}}) > \mathbb{E}[U(X_{\text{wait}})] \text{ if and only if, } X_{\text{sell}} > \mathbb{L}[X_{\text{wait}}].$$

□

Corollary 2.1. *Decision Rule Therefore the decision rule for the liquidation decision problem reduces to*

$$\textit{sell if } X_{\text{sell}} > \mathbb{L}[X_{\text{wait}}].$$

This shows exactly where the illiquidity discount should live in the model: the relevant comparison is not spot versus expected future value, but spot versus *certainty equivalent liquidity.* The gap

$$\mathbb{E}[X_{\text{wait}}] - \mathbb{L}[X_{\text{wait}}]$$

is a risk adjustment that captures aversion to uncertainty over the redemption horizon.

2.2 Exponential Utility Function

A common utility function is the exponential utility, which is given by

$$U(x) = -e^{-\gamma x},$$

where γ is the risk aversion parameter. The higher the γ, the more risk averse the investor is.

The intuition behind the exponential utility is that, losing \$10 when you have \$100 is not the same as losing \$10 when you have \$1000. The exponential utility captures this intuition, because $U(90) - U(100) > U(990) - U(1000)$:

$$U(90) - U(100) = -e^{-\gamma \cdot 90} + e^{-\gamma \cdot 100} > -e^{-\gamma \cdot 990} + e^{-\gamma \cdot 1000} = U(990) - U(1000).$$

See section 2.2 in [5] for an introduction on utility functions in the context of financial pricing.

The next proposition give us an explicit formula for the certainty equivalent liquidity under the exponential utility function.

Proposition 2.2. *Under the exponential utility, the certainty equivalent liquidity can be computed as*

$$\begin{aligned}
\mathbb{L}[X_{\text{wait}}] &= U^{-1}(\mathbb{E}[U(X_{\text{wait}})]) \\
&= -\frac{1}{\gamma}\log(-\mathbb{E}[U(X_{\text{wait}})]) \\
&= -\frac{1}{\gamma}\log\Big(\mathbb{E}\Big[e^{-\gamma X_{\text{wait}}}\Big]\Big).
\end{aligned}$$

3 Asset models

Because RWA prices are stochastic processes, we will evaluate X_{wait} and its expectation $\mathbb{E}[X_{\text{wait}}]$, making use of well known models appropriate for each context. In this section we define these models, and check some of their properties when relevant. The models we are looking at are **Brownian Motion** (BM), **Geometric Brownian Motion** (GBM), **GBM with diffusion jumps**, and **GBM with transient jumps**.

3.1 Brownian Motion

The price of the RWA token (V_t) follows a Brownian motion if it is a solution to the stochastic differential equation below:

$$dV_t = \mu\, dt + \sigma\, dB_t,$$

where $dB_t \sim \mathcal{N}(0, dt)$ for infinitesimal dt, μ is the drift, and σ is the volatility.

While Brownian motion is not a realistic model for asset prices, it is a useful building block for more complex models, and it allows us to illustrate some of the key features of our pricing framework in a simple setting. For example, if X_t follows a Brownian motion, let's compute the liquidity value of waiting for redemption, i.e., $\mathbb{L}[X_{\text{wait}}]$ explicitly.

Proposition 3.1. *Let $Z \sim \mathcal{N}(0,1)$ be a standard normal random variable. Let $\alpha, \beta \in \mathbb{R}$ be arbitrary constants. Then*

$$\mathbb{E}\left[e^{\alpha Z+\beta}\right] = e^{\alpha^2/2+\beta}$$

Proof. The proof requires we look to the density function of the normal distribution, which is given by $f_Z(z) = \frac{1}{\sqrt{2\pi}} e^{-z^2/2}$, and apply the definition of expectation for continuous random variables:

$$\begin{aligned}
\mathbb{E}\left[e^{\alpha Z+\beta}\right] &= e^{\beta}\, \mathbb{E}\left[e^{\alpha Z}\right] \\
&= \frac{e^{\beta}}{\sqrt{2\pi}} \int e^{-z^2/2} e^{\alpha z}\, dz \\
&= \frac{e^{\beta}}{\sqrt{2\pi}} \int e^{-(z^2-2\alpha z)/2}\, dz \\
&= \frac{e^{\beta}}{\sqrt{2\pi}} \int e^{-(z-\alpha)^2/2} e^{\alpha^2/2}\, dz \\
&= e^{\alpha^2/2+\beta}.
\end{aligned}$$

□

If we have X_t, the value of selling the RWA token at time t, to follow a Brownian motion, and using the exponential utility $U(x) = -e^{-\gamma x}$, we can compute the liquidity value of waiting $\mathbb{L}[X_{\text{wait}}]$ as given in proposition 2.2 explicitly:

Proposition 3.2. *Let X be a Brownian motion satisfying the stochastic differential equation $dX_t = \mu\, dt + \sigma\, dB_t$. Then*

$$\mathbb{L}[X] = \mu t - \frac{\gamma \sigma^2 t}{2}.$$

Proof.

$$\begin{aligned}
\mathbb{L}[X] &= -\frac{1}{\gamma} \log\Big(\mathbb{E}\Big[e^{-\gamma X}\Big]\Big) \\
&= -\frac{1}{\gamma} \log\Big(\mathbb{E}\Big[e^{-\gamma(\mu t+\sigma\sqrt{t}Z)}\Big]\Big) \\
&= -\frac{1}{\gamma} \log\Big(e^{\gamma^2\sigma^2 t/2-\gamma\mu t}\Big) \\
&= \mu t - \frac{\gamma\sigma^2 t}{2}.
\end{aligned}$$

□

This shows the trade-off between expected return and risk in a natural way. The liquidity value of waiting decreases as volatility σ increases. The risk aversion parameter γ controls how much the investor cares about risk.

Proposition 3.2 shows an explicit formula for $\mathbb{L}[X]$ because in this scenario X is assumed to be a brownian motion. But in general, the computation of the expectation $\mathbb{E}[X]$ required for the derivation has not an analytical solution. In those cases, we can use numerical methods to estimate $\mathbb{L}[X]$. Hence the power our pricing engine specialized in solving this problem.

3.2 Geometric Brownian Motion (GBM)

The price of the RWA token (V_t) follows a Geometric Brownian Motion (GBM) if it is a solution to the stochastic differential equation below:

$$\frac{dV_t}{V_t} = \mu\, dt + \sigma\, dB_t,$$

where μ is the drift, σ is the volatility, and B_t is a standard Brownian motion.

4 Geometric Brownian Motion with Two-Sided Jump–Diffusion

The price of the RWA token (V_t) follows a two-sided jump–diffusion model if it is a solution to the stochastic differential equation below:

$$\frac{dV_t}{V_{t^-}} = \mu\, dt + \sigma\, dB_t + dJ_t^+ + dJ_t^-.$$

where μ is the drift, σ is the volatility, dB_t is a standard Brownian motion, and dJ_t^+ and dJ_t^- are the positive and negative jump components, respectively. More precisely:

- $dJ_t^+ = (Y_+ - 1)\, dN_t^+, \qquad dJ_t^- = (Y_- - 1)\, dN_t^-,$
- $Y_+ \geq 1$ is the positive-jump multiplier,
- $0 < Y_- \leq 1$ is the negative-jump multiplier.

Where N_t^+ and N_t^- are independent Poisson processes with intensities λ_+ and λ_-, with

$$N_t^+ \sim \text{Poisson}(\lambda_+), \qquad N_t^- \sim \text{Poisson}(\lambda_-),$$

Remarks:

- λ_+, λ_-: expected positive/negative jump counts per day,
- $1 - e^{-\lambda_\pm}$: probability of at least one $\pm$ jump in one day,

(V_{t^-}) is the left limit of the process at time (t) (the value just before time (t)).

Since (V_t) can jump instantly at (t), $V_{t^-} \neq V_t$ when a jump occurs at t. If no jump at (t), then $(V_{t^-} = V_t)$.

The notation $\frac{dV_t}{V_{t^-}}$ indicates that the jump is applied as a return relative to the pre-jump level.

4.1 Depeg Risk Example

Let's use our pricing engine to illustrate how a two-sided jump–diffusion model can help us decide whether to sell immediately or wait for redemption when the RWA is a stablecoin with depeg risk and delayed redemption. Then, let's compute the same example, but with a simple GBM model, and see how the liquidity value of waiting for redemption are too aggressive in the GBM case.

Example 4.1. Consider a stablecoin, with a redemption delay of 30 days, and a risk of depegging during the redemption in the next 30 days. Let's assume the probability of depegging is 10%, and that the depeg is of 5% if it occurs. We can model this scenario with a two-sided jump–diffusion model, with the following parameters:

- $\mu_{\text{daily}} = 0$ (because it's a stablecoin, we expect no drift in the price),
- $\sigma_{\text{daily}} = 0.0015$ (daily volatility of 0.15%),
- $\lambda_- = -\log(1 - 0.10)/30$ (10% probability of depegging over 30 days),
- $\log Y_- = \log(1 - 0.05)$ (depeg of 5% when a negative jump occurs),

Figure 5 shows a simulated path for the price of the stablecoin under this model.
Assume the slippage in the market for this asset is of 20%, i.e., $X_{\text{sell}} = 0.8$.
Let's assume a value of the risk aversion ($\gamma = 60$)[2]. Finally, let's compute

$$\mathbb{L}[X_{\text{wait}}]$$

for this example, and compare it with the value of selling immediately in the market, X_{sell}.

Using our pricing engine, we can get

$$\mathbb{L}[X_{\text{wait}}] = 0.86.$$

This means that, such investor, in this situation, would prefer to wait for redemption rather than sell immediately, since $X_{\text{sell}} = 0.8 < 0.86 = \mathbb{L}[X_{\text{wait}}]$.

Example 4.2. Now, let's look at the data from figure 5 and calibrate a simple GBM model to it. After calibrating we get the following parameters:

- $\mu_{\text{daily}} = 0$ (hard coded zero drift because it's a stablecoin),
- $\sigma_{\text{daily}} = 0.018$ (daily volatility of 1.8%).

Finally, when computing the liquidity value of waiting for redemption under the GBM model, we get $\mathbb{L}[X_{\text{wait}}] = 0.79$, which is lower than the value of selling immediately in the market, $X_{\text{sell}} = 0.8$. This means that, under the GBM model, the investor would prefer to sell immediately rather than wait for redemption, which is not the case when we use the two-sided jump–diffusion model.

This illustrates how a simple GBM model can be too aggressive in pricing uncertainty, and how a more complex model can capture the risk of depegging more accurately.

Table 2 summarizes the parameters and results for both examples.

[2] We can compute the implied risk aversion parameter γ by asking the question 'Given a risky investment that pays $\pm 2\%$, what is the risk aversion parameter γ if the investor is willing to pay 1% to have a sure payoff instead of participating in the risky investment?'

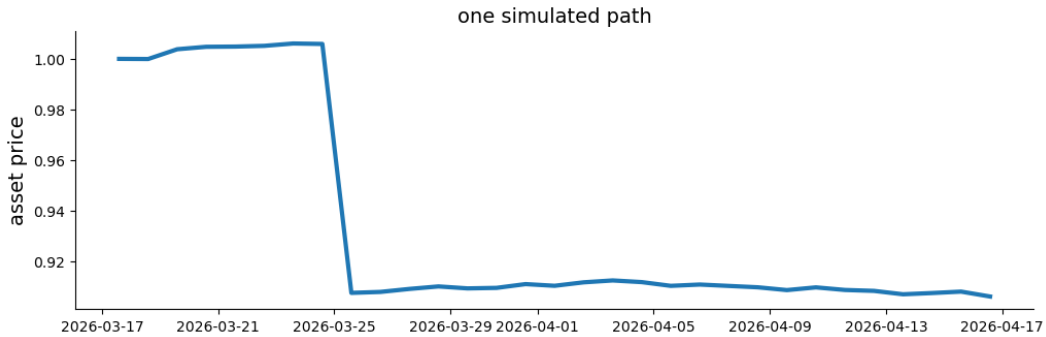


Figure 5: Simulated path for a two-sided jump–diffusion model with depeg. Note the drop of price around march of 2026, which is the simulated depeg event.

Model	μ_{daily}	σ_{daily}	Jump parameters	γ	X_{sell}	$\mathbb{L}[X_{\text{wait}}]$	Decision
Jump–Diffusion	0	0.0015	10% prob. of depeg	60	0.8	0.86	Wait
GBM	0	0.018	None	60	0.8	0.79	Sell

Table 2: Comparison of the depeg-risk example under Jump–Diffusion and GBM. When Brownian motion is used, the calibrated volatility is higher than the diffusion volatility in the jump–diffusion model, because it incorporates the swing caused by the drop, artificially decreasing the liquidity value of waiting for redemption ($\mathbb{L}[X_{\text{wait}}]$). This is one advantage of Jump–Diffusion models.

5 Pricing-engine architecture

The pricing engine is designed to compute the certainty equivalent liquidity $\mathbb{L}[X_{\text{wait}}]$ for a given RWA token in a modular way. That means, the pricer is designed to be modular in terms of models and utility functions. The main components of the pricer are:

1. **Risk aversion calibration**: estimate investor risk preferences from observed behavior or stated utility.
2. **Utility functions**: Exponential utility and other common utility functions to capture different risk preferences. See appendix B for the derivation of the exponential utility function.
3. **Asset models**: stochastic models for the underlying asset over the redemption horizon (Geometric Brownian motions, Jump diffusion, etc.).
4. **Simulation engine**: compute expected utility of the underlying asset through Monte Carlo simulation or numerical integration, depending on the model.

6 Conclusion

In these notes we proposed a utility-based framework for pricing liquidity in tokenized RWAs with delayed redemption. The key idea is that the relevant liquidation comparison is not between the spot sale value and the expected redemption payoff, but between the immediate sale value X_{sell} and the certainty equivalent liquidity $\mathbb{L}[X_{\text{wait}}]$. This reformulation makes the investor's risk preferences explicit and provides a natural way to translate redemption uncertainty into an economically meaningful liquidity discount.

We showed that this framework leads to intuitive pricing behavior. Under exponential utility, the liquidity value of waiting falls as volatility rises, matching the idea that a risk-averse investor should demand compensation for uncertainty over the redemption horizon. We also illustrated that model choice matters materially: in the depeg example, a jump–diffusion model captures tail events more faithfully than a simple GBM calibration, leading to a different liquidation decision. This is particularly relevant for DeFi RWAs, where redemption frictions, discontinuous repricings, and protocol-specific risks are often first-order features rather than small corrections.

From an implementation perspective (available in *Dope*[3]), the framework is modular. Risk aversion calibration, utility specification, asset modeling, and simulation can be treated as separate components inside a pricing engine, making the approach adaptable to different investor types and different classes of RWAs. In practice, this means the same architecture can support assets with relatively smooth dynamics as well as instruments exposed to jump risk, depegs, or episodic liquidity stress.

[3] https://github.com/ipor-labs/dope

References

[1] OpenEden, "Redemptions," https://docs.openeden.com/tbill/redemptions, 2025, accessed: 2026-03-11.

[2] USD.AI, "Mechanism design," https://docs.usd.ai/3.-qev-redeem#mechanism-design, 2026, accessed: 2026-03-11.

[3] Ondo Finance, "Ousg token overview," https://docs.ondo.finance/qualified-access-products/ousg/overview, 2026, accessed: 2026-03-11.

[4] USD.AI, "Unstaking and withdrawals," https://docs.usd.ai/faq/usdai-and-susdai-101#unstaking-and-withdrawals, 2025, accessed: 2026-03-11.

[5] R. A. Carmona, Ed., *Indifference Pricing: Theory and Applications*. Princeton University Press, 2008.

[6] K. J. Arrow, *Essays in the Theory of Risk-Bearing*. Amsterdam: North-Holland, 1970.

[7] J. W. Pratt, "Risk aversion in the small and in the large," *Econometrica*, vol. 32, no. 1/2, pp. 122–136, 1964.

[8] R. T. Rockafellar and S. Uryasev, "Optimization of conditional value-at-risk," *Journal of Risk*, vol. 2, no. 3, pp. 21–41, 2000.

A Appendix - Code Usage Example

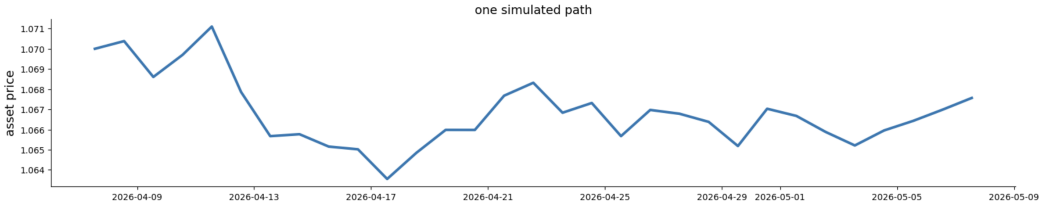
The figure below shows a code example of how the pricer can be used with *Dope*[4] to evaluate a RWA token with delayed redepmtion:

```python
from dope.rwa import RWA

rwa = RWA(
    V0=1.07,
    gamma=gamma,
    model="gbm",
    model_params={
        "mu": -0.0001*365.25,
        "sigma": 0.0015*np.sqrt(365.25),
    },
    seed=42,
)
gbm_path = rwa.simulate_path_with_timestamps(n_days=30)


decision = rwa.decide(n_days=30, slippage=0.005)

print("Decision:", decision["decision"])
print("I sell if I get more than         :", decision["L_wait"])
print("Current swap price (with slippage):", decision["X_sell"])
```




```
Decision: SELL
I sell if I get more than         : 1.064444583300873
Current swap price (with slippage): 1.06465
```

B Appendix - Utility Function Derivations

In this section we derive the risk aversion parameter γ

Definition B.1. Let current wealth be $w > 0$. Suppose the investor faces a small gamble ε with expected value zero:

$$\mathbb{E}[\varepsilon] = 0.$$

In this case, the investor's expected utility is given by

$$\mathbb{E}[U(w + \varepsilon)].$$

With this setup, we have what we need to define γ.

Proposition B.1. *Assume U is an utility function twice differentiable. Then, the following approximation holds:*

$$\mathbb{E}[U(w + \varepsilon)] \approx U(w) + \frac{1}{2}U''(w)\mathrm{Var}(\varepsilon).$$

Proof. Using the second-order Taylor expansion around w,

$$U(w + \varepsilon) \approx U(w) + U'(w)\varepsilon + \frac{1}{2}U''(w)\varepsilon^2.$$

[4] https://github.com/ipor-labs/dope

Taking expectations gives

$$\mathbb{E}[U(w+\varepsilon)] \approx U(w) + U'(w)\mathbb{E}[\varepsilon] + \frac{1}{2}U''(w)\mathbb{E}[\varepsilon^2].$$

Since $\mathbb{E}[\varepsilon] = 0$ and $\mathbb{E}[\varepsilon^2] = \mathrm{Var}(\varepsilon)$,

$$\mathbb{E}[U(w+\varepsilon)] \approx U(w) + \frac{1}{2}U''(w)\mathrm{Var}(\varepsilon).$$

□

Definition B.2. The absolute risk premium π is the sure amount the investor is willing to pay to avoid the gamble ε, defined implicitly by

$$U(w-\pi) = \mathbb{E}[U(w+\varepsilon)].$$

Proposition B.2. *Under the assumptions above,*

$$\pi \approx -\frac{1}{2}\frac{U''(w)}{U'(w)}\mathrm{Var}(\varepsilon).$$

Proof. Expand the left-hand side to first order (Taylor expansion):

$$U(w-\pi) \approx U(w) - U'(w)\pi.$$

Using proposition B.1,

$$U(w) - U'(w)\pi \approx U(w) + \frac{1}{2}U''(w)\mathrm{Var}(\varepsilon).$$

Cancel $U(w)$ on both sides and solve for π:

$$-U'(w)\pi \approx \frac{1}{2}U''(w)\mathrm{Var}(\varepsilon),$$

hence

$$\pi \approx -\frac{1}{2}\frac{U''(w)}{U'(w)}\mathrm{Var}(\varepsilon).$$

□

Definition B.3. The coefficient

$$A(w) := -\frac{U''(w)}{U'(w)}$$

is the Arrow–Pratt coefficient of absolute risk aversion (from [6] and [7]). Thus

$$\pi \approx \frac{1}{2}A(w)\mathrm{Var}(\varepsilon).$$

To finish this section, we prove that the risk aversion parameter γ we used throughout the text, is $A(w)$:

Proposition B.3. *If $U(w) = -e^{-w\gamma}$, then $\gamma = A(w)$.*

Proof. First, note that $U'(w) = \frac{d(-e^{-w\gamma})}{dw} = -\gamma e^{-w\gamma}$, and $U''(w) = \gamma^2 e^{-w\gamma}$. Therefore,

$$A(w) = -\frac{U''(w)}{U'(w)} = -\frac{\gamma^2 e^{-w\gamma}}{-\gamma e^{-w\gamma}} = \gamma.$$

□

C Appendix - Conditional Value at Risk (CVaR)

While certainty equivalent liquidity captures investor preferences through utility, it is also useful to evaluate downside risk using tail-based risk measures. One of the most widely used measures in risk management is the Conditional Value at Risk (CVaR) (see the seminal paper [8] for an introduction on cVaR optimization).

Definition C.1. (Value at Risk) Let X be a real-valued random payoff. For a confidence level $\alpha \in (0, 1)$, the Value at Risk (VaR) at level α is defined as

$$\mathrm{VaR}_\alpha(X) := \inf\{x \in \mathbb{R} \mid \mathbb{P}(X \leq x) \geq \alpha\}.$$

This represents the worst loss (or lowest payoff) not exceeded with probability α.

Definition C.2. (Conditional Value at Risk)

An alternative method for assessing the risk of waiting for redemption is to use tail-based risk measures, such as the Conditional Value at Risk (CVaR). CVaR focuses on the expected payoff in the worst-case scenarios beyond a certain confidence level.

The Conditional Value at Risk at level α, denoted $\mathrm{CVaR}_\alpha(X)$, is defined as the expected value of the payoff in the worst α-tail:

$$\mathrm{CVaR}_\alpha(X) := \mathbb{E}[X \mid X \leq \mathrm{VaR}_\alpha(X)].$$

Equivalently, CVaR captures the average outcome in the worst-case scenarios beyond VaR.

Interpretation in the liquidation problem In our framework, X_{wait} represents the uncertain payoff from waiting through the redemption period. While the expected value $\mathbb{E}[X_{wait}]$ and the certainty equivalent $L[X_{wait}]$ summarize average and utility-adjusted outcomes, respectively, CVaR provides a direct measure of downside risk.

In particular:

- $\mathbb{E}[X_{wait}]$: average payoff,
- $L[X_{wait}]$: utility-adjusted payoff (risk-sensitive),
- $\mathrm{CVaR}_\alpha(X_{wait})$: average payoff in worst-case scenarios.

This is especially relevant for RWAs with jump risk (e.g., depegs), where tail events dominate investor concerns.

Decision rule with CVaR A risk-constrained investor may incorporate CVaR into the liquidation decision. For example, define a minimum acceptable downside level ($\underline{X}$). Then the investor prefers to wait only if:

$$\mathrm{CVaR}_\alpha(X_{wait}) \geq \underline{X}.$$

Alternatively, CVaR can be combined with the certainty equivalent into a hybrid rule:

$$\text{Sell if } X_{sell} > L[X_{wait}] \quad \text{or} \quad \mathrm{CVaR}_\alpha(X_{wait}) < X_{sell}.$$

This captures both average risk-adjusted value and tail risk exposure.